UNIVERSIDAD DEL NORTE

Facultad de Ingeniería

Carrera: Ingeniería en Informática



Asignatura: Programación IV

Docente: Prof. Marcos Arrieta Moreira

Menina Coiffure

Proyecto Integrador · Avance 4: documento final

Cierre metodológico, benchmarking de las arquitecturas A y B, HAProxy Stats, despliegue en producción, ethical hacking y documentación técnica

Grupo N° 5

Giovanni Manuel Rojas Giménez
Líder · Backend & Software Architect Lead · DevOps & Cloud Engineer

Braihan de Jesús Vega Cardozo
Backend & Software Architect Lead · DevOps & Cloud Engineer

Noelia Magalí Servián Lezcano
Backend & Software Architect Lead · DevOps & Cloud Engineer

Danna Luján Sanabria Prieto
Scrum Master / Project Manager · Frontend & QA Engineer

Thiago Ezequiel Rodríguez Sánchez
Scrum Master / Project Manager · Frontend & QA Engineer

Gabriela Luján Ramírez Bogado
Scrum Master / Project Manager · Frontend & QA Engineer

Asunción, Paraguay

Fecha de entrega: 13/11/2026

Tabla de contenido

1. Introducción y Objetivos 4
   1.1. Contexto empresarial 4
   1.2. Entregables del Avance 4 4
   1.3. Qué cambió desde el Avance 3 4
2. Alcance Final del Proyecto 6
   2.1. Dentro del alcance (In-Scope) 6
   2.2. Fuera del alcance (Out-of-Scope) 6
3. Cierre Metodológico 7
   3.1. Tablero y criterio de actualización 7
   3.2. Cierre del Sprint 4 7
   3.3. Sprint 5 (31/10 – 13/11) 7
   3.4. Flujo acumulado del Sprint 5 8
   3.5. Métricas de todo el proyecto 9
   3.6. Sprint Review final 9
   3.7. Retrospectiva final 9
   3.8. Evidencia de ceremonias 10
4. Arquitectura de la Solución 11
   4.1. Tecnologías 11
   4.2. Funcionalidades del sistema 11
   4.3. Reglas de negocio 13
   4.4. Sesiones centralizadas 14
   4.5. Entornos de desarrollo y producción 14
5. Balanceo de Carga y Arquitecturas A y B 15
   5.1. Arquitecturas A y B 15
   5.2. Configuración final de HAProxy 15
   5.3. Failover 16
6. Pruebas de Rendimiento (Benchmarking) 17
   6.1. Objetivo y método 17
   6.2. Resultados 17
   6.3. Análisis 18
   6.4. Repetición en el VPS (trabajo futuro) 19
7. Dashboard HAProxy Stats con Tráfico Real 20
8. Despliegue en Producción 22
   8.1. Arquitectura de producción 22

8.2. Componentes 22
8.3. Procedimiento 23
8.4. Estado del despliegue 23
9. Seguridad 24
9.1. Hallazgos principales y correcciones 24
9.2. Verificación 24
10. Evidencia de Pruebas 26
10.1. Pruebas automatizadas 26
10.2. Pruebas sobre la infraestructura 26
11. Repositorio Git y Documentación 27
11.1. Repositorio 27
11.2. Estructura 27
11.3. Documentación disponible 27
12. Ethical Hacking | Test de Intrusión Externo | Web 28
12.1. Objetivo 28
12.2. Alcance 28
12.3. Personal comprometido 28
12.4. Duración 28
12.5. Herramientas utilizadas 28
12.6. Contexto del test de intrusión 29
12.7. App Web Institucional 30
12.8. App Web de Clientas 30
12.9. Resultados 31
13. Conclusiones 32
14. Referencias 33

1. Introducción y Objetivos

1.1. Contexto empresarial

Menina Coiffure es una peluquería y centro de estética familiar de Villa Elisa, en el departamento Central. La fundó la Sra. Mónica en 2018 y hoy trabajan ahí tres personas: ella, que además de atender maneja la agenda, una estilista y una asistente. Atienden en el local y también a domicilio. Cuando empezamos a investigar el negocio, todas las citas se anotaban en cuadernos o quedaban en chats de WhatsApp: a veces se pisaban dos turnos, se daban citas fuera del horario y no quedaba registrado cuánto se le cobró realmente a cada clienta.

Para resolverlo construimos una aplicación web con Laravel 13 y PostgreSQL 18, con la interfaz en Vue 3 e Inertia, que corre en contenedores con Docker Compose y se administra con Portainer. HAProxy reparte el tráfico entre dos nodos de la aplicación (las arquitecturas A y B). Este documento es la entrega final del proyecto: cierra la parte metodológica, presenta las pruebas de rendimiento, el panel de HAProxy con tráfico real y el test de intrusión externo, y resume la documentación técnica del sistema tal como queda en producción.

1.2. Entregables del Avance 4

La consigna pide cuatro entregables para este avance. Esta tabla indica dónde está cada uno, junto con el test de intrusión que se sumó al final:

Entregable exigido	**Dónde está en este documento**
Cierre metodológico (Sprint Review / Retrospective final o métricas de flujo)	Sección 3: Sprint 5, flujo acumulado, métricas de todo el proyecto, Sprint Review y Retrospectiva final
Pruebas de rendimiento (Benchmarking RPS / latencia, Arquitectura A vs. B)	Sección 6
Dashboard HAProxy Stats (puerto 8404) con tráfico real	Sección 7
Repositorio Git y documentación técnica y metodológica completa	Secciones 4, 5, 8, 9, 10 y 11
Ethical Hacking: test de intrusión externo y web	Sección 12

1.3. Qué cambió desde el Avance 3

Después del Avance 3 (30/10) el sistema pasó de funcionar en una PC a estar publicado en producción. Los cambios principales fueron:

Tema	**Avance 3 (30/10)**	**Avance 4 (13/11)**
Inicio de sesión	KeyCloak 26 (servidor de identidad aparte)	Autenticación propia de Laravel con las cuentas en PostgreSQL
Avisos a la dueña	Telegram	Correo electrónico por Brevo
Entorno	Solo desarrollo (Docker Compose local)	Desarrollo y producción separados; producción lista para un VPS de Contabo con HTTPS

Tema	Avance 3 (30/10)	Avance 4 (13/11)
Seguridad	Sin auditar	Auditoría completa en tres frentes y 4 tandas de correcciones (sección 9)
Pruebas automatizadas	191 pruebas	232 pruebas (1.655 aserciones)
Rendimiento	Failover probado	Benchmark A vs. B vs. A+B con 81.000 peticiones y 0 errores (sección 6)
Imagen	Logo provisorio	Logo "Sello M" elegido por la Sra. Mónica

2. Alcance Final del Proyecto

El alcance se definió en el Avance 1 y se ajustó durante el proyecto con la dueña. Esta es la declaración de alcance tal como queda al cierre:

2.1. Dentro del alcance (In-Scope)

Módulo	Qué incluye	Estado
Agenda de reservas	Crear, modificar y cancelar citas en el salón o a domicilio (con traslado), con las 5 reglas de negocio de la sección 4.3	Terminado
Clientas	Ficha, preferencias, alergias e historial de visitas	Terminado
Catálogo	Servicios, categorías y precios con historial (cada cita guarda el precio de su fecha)	Terminado
Personal	Fichas, especialidades y permisos por profesional	Terminado
Pagos	Seña, pago final, saldo, métodos de pago y promociones por día	Terminado
Reportes	Reservas, servicios más pedidos e ingresos por período	Terminado
Página pública y cuentas de clientas	Catálogo, horario y promociones; la clienta crea su cuenta y pide citas que la dueña acepta o rechaza	Terminado
Cuentas y seguridad	Inicio de sesión por rol, registro, recuperación de contraseña por correo, límite de intentos	Terminado
API REST	Toda la funcionalidad expuesta como API JSON, documentada con OpenAPI/Swagger (solo para la administración)	Terminado
Infraestructura	Docker Compose, Portainer, HAProxy con dos arquitecturas, sesiones centralizadas y despliegue en producción	Terminado (publicado en el VPS el 12/11)

2.2. Fuera del alcance (Out-of-Scope)

- **Inventario de insumos (HU-15 a HU-18):** descartado el 22/10 a pedido de la dueña; por ahora el salón no necesita controlar el stock.
- **Turnos fijos del personal (HU-08):** reemplazado por un horario único del salón (T-09), porque las profesionales atienden según la demanda.
- **Facturación electrónica y cobro en línea:** el sistema registra los pagos, pero no emite facturas ni cobra con tarjeta.
- **Aplicación móvil nativa:** el sistema es web y se adapta al celular.

3. Cierre Metodológico

3.1. Tablero y criterio de actualización

Trabajamos con Scrum durante todo el semestre, con el tablero en Trello en vista Kanban y estas columnas: Product Backlog, Sprint Backlog, En progreso, En revisión / pruebas y Terminado. Los sprints duraron entre 2 y 3 semanas.

Igual que en el Avance 3, las fechas de cada tarjeta salen del historial de Git, porque cualquiera lo puede comprobar: **En progreso** es el día del primer commit de la tarea, **En revisión** el día del último commit y **Terminado** el día en que se unió a `main` con un Pull Request. Las tareas que no dejan commits (por ejemplo, la configuración del correo en Brevo) se registran en la descripción de la tarjeta.

Sprint	Período	Duración	Objetivo
Sprint 1	01/09 – 14/09	2 semanas	Autenticación por rol, catálogo, personal y migraciones
Sprint 2	15/09 – 25/09	1,5 semanas	Reservas, agenda, sesiones centralizadas y Portainer
Sprint 3	26/09 – 16/10	3 semanas	Frontend del panel y API completa con roles
Sprint 4	17/10 – 30/10	2 semanas	Clientas, configuración, HAProxy, arquitecturas A y B y failover
Sprint 5	31/10 – 13/11	2 semanas	Pase a producción: cuentas propias, correo, despliegue y seguridad

3.2. Cierre del Sprint 4

En el Avance 3 el Sprint 4 tenía 13 de 16 tarjetas terminadas al 28/10. Las tres restantes se cerraron el 30/10: HAProxy con las arquitecturas A y B (T-14) y la demostración de failover (T-15) se unieron a `main` con el Pull Request #12, y el documento del Avance 3 (T-16) se entregó ese mismo día. El Sprint 4 cerró con **16 de 16 tarjetas terminadas (100 %)**.

3.3. Sprint 5 (31/10 – 13/11)

El objetivo del Sprint 5 fue dejar el sistema listo para producción. Al revisar qué hacía falta para subirlo a un servidor, decidimos con el equipo sacar KeyCloak (necesitaba un servidor propio con bastante memoria y salía caro mantenerlo) y reemplazar los avisos de Telegram por correo. Las fechas son las del historial de Git:

Código	Tarea	Responsable	En progreso	Terminado	Evidencia
T-17	Reemplazo de KeyCloak por la autenticación de Laravel	Giovanni	11/11	11/11	PR #14
T-18	Avisos por correo en lugar de Telegram	Giovanni	11/11	11/11	PR #14
T-19	README actualizado para el Avance 4	Thiago	10/11	10/11	PR #13

Código	Tarea	Responsable	En progreso	Terminado	Evidencia
T-20	Preparación para producción (imágenes, Caddy, scripts, catálogo real)	Giovanni	11/11	11/11	PR #14
T-21	Bandeja de pedidos de cita: 3 visibles y hasta 50	Giovanni	11/11	11/11	PR #14
T-22	Logo "Sello M" elegido por la dueña	Thiago	12/11	12/11	PR #15
T-23	Auditoría y blindaje de seguridad (4 tandas)	Giovanni	12/11	12/11	PR #16
T-24	Despliegue en el VPS de Contabo y guía de despliegue	Giovanni	12/11	12/11	PR #17 y sitio publicado
T-25	Benchmark local A vs. B vs. A+B	Giovanni	12/11	12/11	Sección 6
T-26	Documento del Avance 4	Thiago · Giovanni	12/11	13/11	Este documento

El Sprint 5 cerró el 13/11 con 10 de 10 tarjetas terminadas (100 %). El benchmark local terminó el 12/11 y este documento, el 13/11, con la entrega.

3.4. Flujo acumulado del Sprint 5

La tabla muestra cuántas tarjetas había en cada columna al final de cada día hábil (SB = Sprint Backlog, EP = En progreso, ER = En revisión, T = Terminado):

Día	31/10	02/11	03/11	04/11	05/11	06/11	09/11	10/11	11/11	12/11
SB	10	10	10	10	10	10	10	9	5	0
EP	0	0	0	0	0	0	0	0	0	2
ER	0	0	0	0	0	0	0	0	0	0
T	0	0	0	0	0	0	0	1	5	8

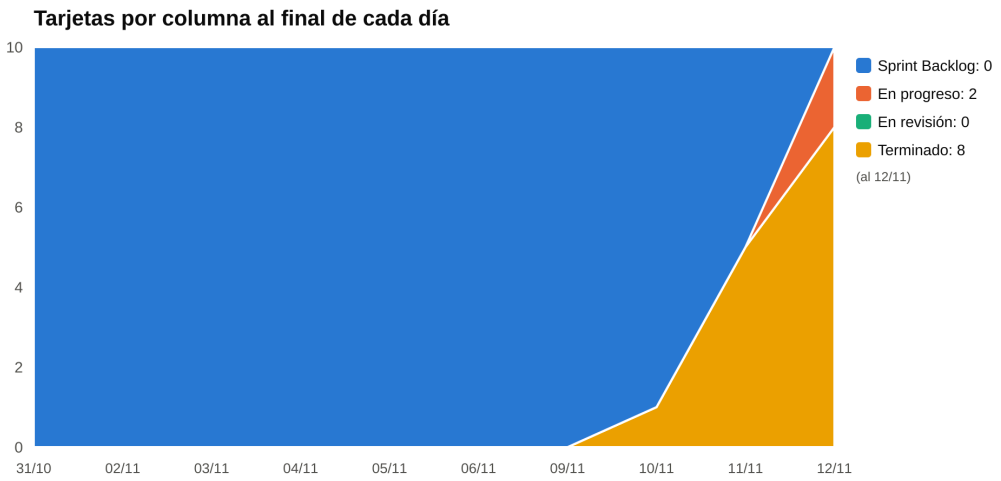


Figura 1. Diagrama de Flujo Acumulado del Sprint 5 (datos de Git al 12/11/2026).

El gráfico muestra dos cosas. La buena: la columna **En revisión** nunca acumuló tarjetas, porque desde el Sprint 4 cada tarea se une a `main` con su propio Pull Request apenas pasa la revisión. La que hay que mejorar: entre el 31/10 y el 09/11 no se integró nada y las 8 tarjetas terminadas se concentraron en tres días (10, 11 y 12/11). Es el mismo efecto de cierre que vimos en el Sprint 2.

3.5. Métricas de todo el proyecto

Sprint	Tarjetas terminadas	Máximo simultáneo en revisión	Cycle Time promedio
Sprint 1	3 de 4 (75 %)	1	6,3 días
Sprint 2	5 de 7 (71 %)	3	7,0 días
Sprint 3	9 de 14 (64 %)	6	8,0 días
Sprint 4	16 de 16 (100 %)	5	2,6 días
Sprint 5	10 de 10 (100 %)	0	Menos de 1 día (*)

(*) En el Sprint 5 cada tarea se integró el mismo día de su primer commit. Como Git no registra cuándo se empezó a pensar o investigar una tarea, ese valor subestima el tiempo real. Los Cycle Time de los Sprints 1 y 2 los calculamos con los movimientos de tarjetas del informe del Avance 2.

Lectura de las métricas. El cambio más claro del proyecto vino en el Sprint 4, cuando pasamos a un Pull Request por tarea: el Cycle Time bajó de 8,0 días en el Sprint 3 a 2,6 en el Sprint 4, y el máximo de tarjetas esperando revisión pasó de 6 en el Sprint 3 a 0 en el Sprint 5. El WIP Limit de 2 tarjetas en revisión que propusimos en el Avance 2 recién se cumplió en el Sprint 5. El problema que se repite en casi todos los sprints es concentrar el trabajo al final.

3.6. Sprint Review final

Dato	Detalle
Fecha	12/11/2026
Modalidad	Asíncrona por WhatsApp
Asistentes	Los seis integrantes del equipo
Incremento mostrado	Sistema completo con cuentas propias y avisos por correo, despliegue en producción con HTTPS, balanceo A/B con failover, panel de HAProxy con tráfico real y resultados del benchmark
Comentarios de la dueña	La dueña participó durante el proyecto a través de Noelia: pidió sacar el inventario y reemplazar los turnos por el horario único del salón (Sprint 4), y eligió el logo «Sello M» entre cinco propuestas (Sprint 5).
Cambios al backlog	Sin tarjetas nuevas. Se retiraron KeyCloak y Telegram (T-17 y T-18) y se sumó el cambio de logo (T-22).

3.7. Retrospectiva final

La Retrospectiva repasó los cinco sprints y no solo el último. La hicimos de forma asíncrona por WhatsApp el 12/11, con tres columnas: qué funcionó, qué mejorar y qué acción tomaríamos en un próximo proyecto.

Qué funcionó	Qué mejorar	Acciones
Un Pull Request por tarea: dejó de acumularse trabajo en revisión (de 6 tarjetas a 0) y bajó el Cycle Time	Concentración del trabajo al cierre de los sprints (Sprints 2 y 5)	Repartir las tarjetas a lo largo del sprint y no dejar entrar una nueva a revisión si ya hay dos esperando (WIP Limit de 2)
Fechas del tablero respaldadas por Git: cualquiera puede verificarlas	Tarjetas sin commits (configuraciones, reuniones) que no quedan registradas solas	Anotar en la tarjeta de Trello la fecha real de inicio y de cierre cuando la tarea no deja commits
Pruebas automatizadas en cada cambio: de 19 pruebas en el Avance 2 a 232	Ceremonias registradas en forma asíncrona y con poco detalle	Dejar un acta breve de cada ceremonia (fecha, asistentes y acuerdos) en un documento fijo
Contacto con la dueña: sacar el inventario y simplificar los turnos antes de invertir más tiempo en algo que no iba a usar	Arquitecturas A y B iguales: la consigna pedía dos arquitecturas distintas para comparar	Definir las diferencias entre arquitecturas en el diseño, antes de construirlas, y medir su rendimiento antes de darlas por cerradas

3.8. Evidencia de ceremonias

Ceremonia	Fecha	Modalidad	Asistentes
Sprint Planning (Sprint 4)	17/10/2026	Asíncrona por WhatsApp	Equipo
Daily Standups (Sprint 4)	17/10/2026 – 30/10/2026	Asíncrona por WhatsApp	Equipo
Sprint Planning (Sprint 5)	31/10/2026	Asíncrona por WhatsApp	Equipo
Daily Standups (Sprint 5)	31/10/2026 – 13/11/2026	Asíncrona por WhatsApp	Equipo
Sprint Review final	12/11/2026	Asíncrona por WhatsApp	Equipo
Retrospectiva final	12/11/2026	Asíncrona por WhatsApp	Equipo

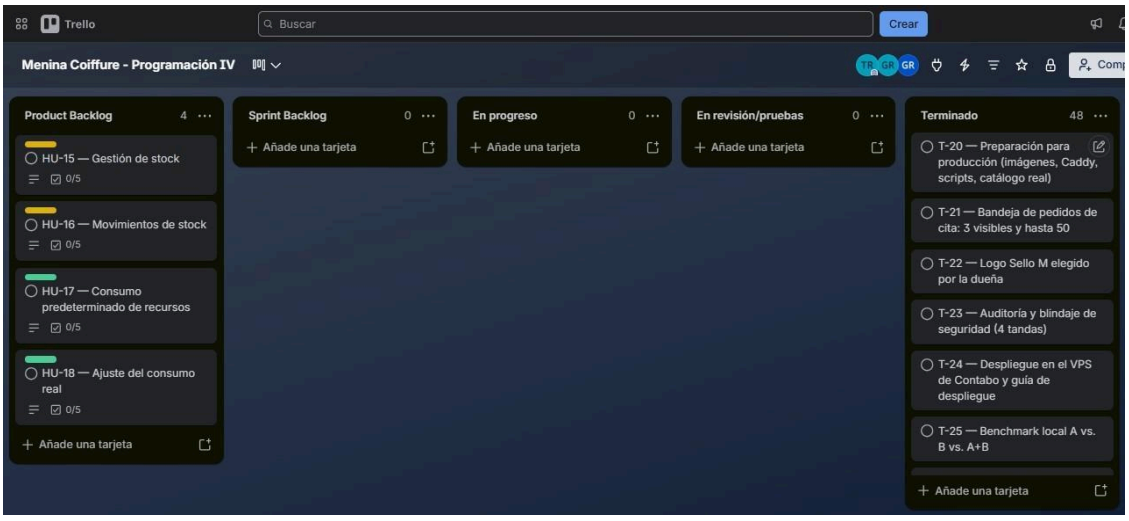


Figura 2. Tablero de Trello al cierre del Sprint 5: las tarjetas del sprint en Terminado y, en el Product Backlog, las historias de inventario (HU-15 a HU-18) que se descartaron.

4. Arquitectura de la Solución

4.1. Tecnologías

Capa	Tecnología
Backend	Laravel 13 sobre PHP 8.4 (PHP-FPM con OPcache en producción)
Base de datos	PostgreSQL 18
Frontend	Vue 3.5, Inertia 3, Tailwind CSS 4 y Vite 8
Cuentas	Autenticación de Laravel: cuentas en PostgreSQL, contraseñas con bcrypt, límite de intentos, "Seguir conectado" y recuperación por correo
Correo	SMTP de Brevo (avisos a la dueña y recuperación de contraseña)
Servidor web	nginx, una instancia por nodo, cada una con su propio PHP-FPM
Balanceador	HAProxy 3.0 con leastconn, health checks y panel de estadísticas
HTTPS (producción)	Caddy 2: certificado de Let's Encrypt automático y redirección de http a https
Contenedores	Docker Compose y Portainer CE
Pruebas y API	PHPUnit, OpenAPI con Swagger UI y ApacheBench para el benchmark

4.2. Funcionalidades del sistema

- **Panel del equipo:** citas del día por profesional, pedidos de las clientas, agenda con filtros, carga de reservas en el salón o a domicilio, pagos con seña y saldo, fichas de clientas, servicios con historial de precios, personal con especialidades, reportes y configuración (horario, días sin atención, permisos, métodos de pago, promociones y especialidades).
- **Página pública del salón** (`/salon`): catálogo, promociones del día, horario y botón de WhatsApp.
- **Cuenta de la clienta** (`/mi-cuenta`): ve sus citas y pide nuevas eligiendo día, hora, servicios y lugar; la dueña recibe un aviso por correo.
- **Roles y permisos:** la administración puede todo; cada profesional ve su propia agenda y la dueña le habilita hasta cuatro permisos más.

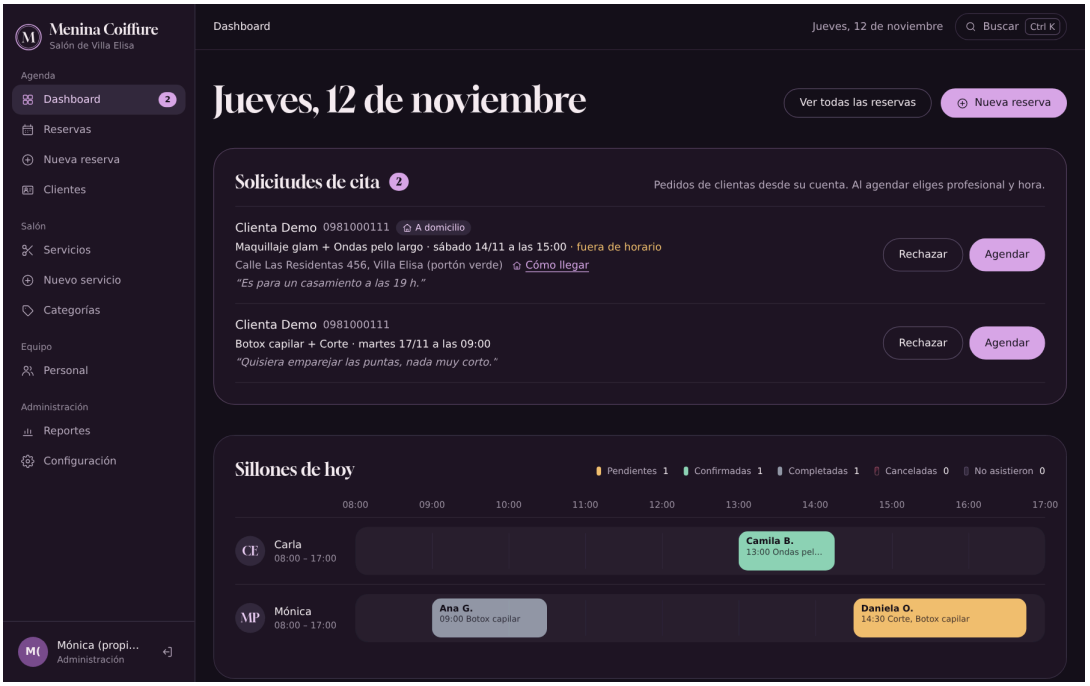


Figura 3. Dashboard de la dueña: pedidos de cita de las clientas y agenda del día por profesional.

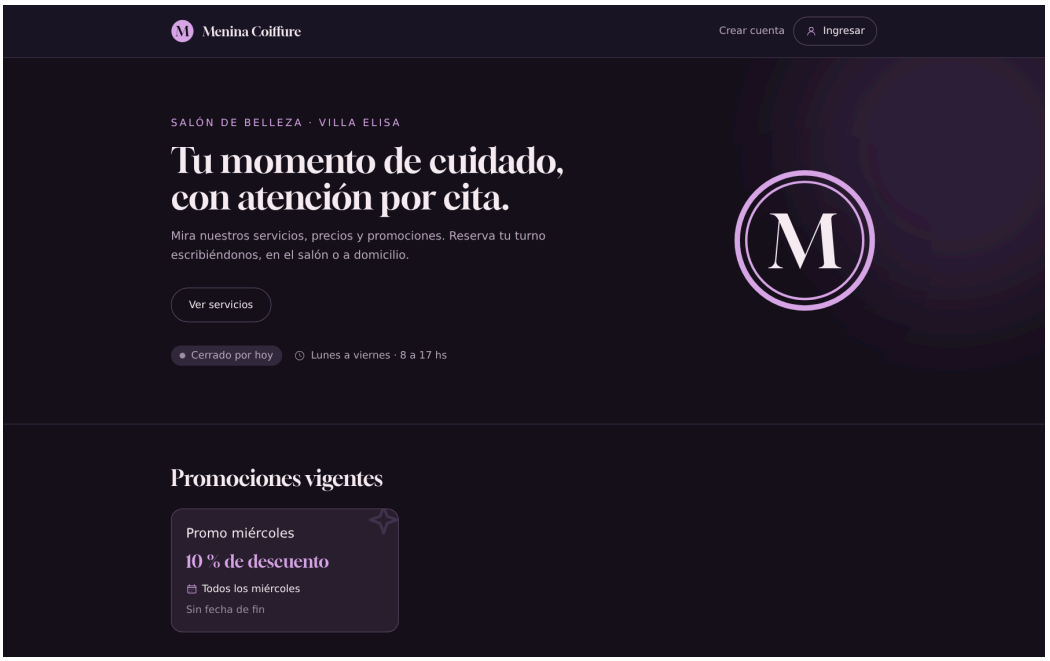


Figura 4. Página pública del salón, con el logo "Sello M".

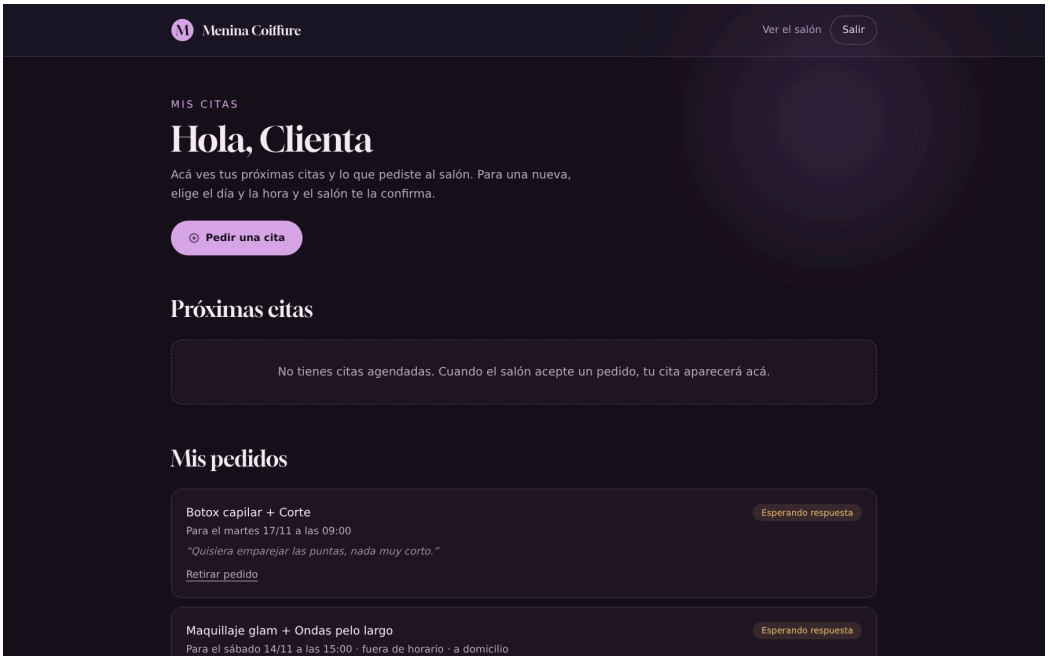


Figura 5. Cuenta de la clienta: sus citas y los pedidos esperando respuesta.

4.3. Reglas de negocio

Toda la lógica de las reservas está en `src/app/Services/ReservaService.php` y corre dentro de una transacción: una reserva se guarda entera o no se guarda nada.

#	Regla	Descripción
1	Sin solapamiento	Una profesional no puede tener dos citas activas superpuestas; a domicilio se suma el tiempo de traslado.
2	Horario del salón	La cita debe caber en el horario del salón, salvo que alguien con permiso la marque como fuera de horario (06:00 a 22:00).
3	Servicios activos	No se puede reservar ni agregar un servicio inactivo.
4	Precio congelado	Cada línea guarda el precio vigente a la fecha de la cita.
5	Estados finales	Una reserva completada o cancelada no se modifica ni se reactiva.
6	Pedidos de clientas (nueva)	Cada clienta puede tener hasta 5 pedidos esperando respuesta y mandar como máximo 10 cada 10 minutos.

Concurrencia entre nodos. Si dos personas reservan a la misma profesional al mismo tiempo y cada pedido cae en un nodo distinto, el sistema bloquea el registro de la profesional hasta terminar la transacción: el segundo pedido espera y, cuando le toca, la Regla 1 lo rechaza. Lo mismo se hace al registrar un pago o modificar una reserva, y aceptar, cancelar o rechazar un pedido solo cambia su estado si sigue pendiente.

4.4. Sesiones centralizadas

Las sesiones, la caché y las colas se guardan en PostgreSQL, que comparten los dos nodos. Así a la persona le da igual qué nodo la atiende: si inicia sesión en A y la siguiente petición cae en B, la sesión sigue abierta. En producción, además, la sesión va cifrada y la cookie solo viaja por HTTPS:

```
SESSION_DRIVER=database
CACHE_STORE=database
QUEUE_CONNECTION=database
SESSION_ENCRYPT=true          # solo producción
SESSION_SECURE_COOKIE=true    # solo producción
```

4.5. Entornos de desarrollo y producción

El mismo repositorio tiene los dos entornos, con archivos de configuración distintos:

	Desarrollo (PC del equipo)	**Producción (VPS)**
Archivo de Compose	`docker-compose.yml`	`docker-compose.prod.yml`
Configuración	`src/.env` (no se sube a Git)	`deploy/.env.production` (se crea en el servidor)
Código	Montado desde la carpeta `src`	Dentro de la imagen (Dockerfile.prod multi-etapa)
Entrada	HAProxy en `127.0.0.1:8000`, sin HTTPS	Caddy en los puertos 80 y 443, con HTTPS
Paneles	HAProxy en `127.0.0.1:8404`, Portainer en `127.0.0.1:9443`	Los mismos, solo en el servidor (se entra por túnel SSH)
Base de datos	`127.0.0.1:5433`, datos de prueba	Sin puerto publicado; catálogo real y usuario sin superusuario
PHP-FPM	5 procesos por nodo	12 procesos por nodo y OPcache sin revisar archivos

5. Balanceo de Carga y Arquitecturas A y B

HAProxy está delante de las dos arquitecturas y decide a cuál mandar cada petición. La configuración, el algoritmo y la demostración de failover se presentaron en detalle en el Avance 3; acá resumimos el diseño y lo que cambió para producción.

5.1. Arquitecturas A y B

Las arquitecturas A y B son dos nodos que corren la misma aplicación. Cada nodo tiene su propio nginx y su propio PHP-FPM, así la carga de PHP se reparte de verdad y un nodo puede caerse sin arrastrar al otro. Los dos comparten la base de datos y, con ella, las sesiones.

Componente	Arquitectura A	Arquitectura B
Nombre en HAProxy	nodo_a	nodo_b
Servidor web	nginx	nginx_b
PHP-FPM	app (host nodo-a)	app_b (host nodo-b)
Base de datos	PostgreSQL (compartida)	PostgreSQL (compartida)

5.2. Configuración final de HAProxy

El backend de producción (`deploy/haproxy/haproxy.cfg`) quedó así:

```
backend nodos_web
    balance leastconn
    option httpchk
    http-check send meth GET uri /up ver HTTP/1.1 hdr Host localhost
    http-check expect status 200
    http-response set-header X-Servido-Por %s if { env(APP_DEMO) -m str true }
    default-server check inter 2s fastinter 1s downinter 5s fall 2 rise 2
                   maxconn 12 resolvers docker init-addr libc,none
    server nodo_a nginx:80
    server nodo_b nginx_b:80
```

Ajuste	Desarrollo (Avance 3)	Producción	Por qué
Algoritmo	leastconn	leastconn	Las peticiones no tardan lo mismo: manda cada una al nodo con menos trabajo en curso
Health check	Cada 1 s	Cada 2 s; 1 s mientras cambia de estado y 5 s si está caído	Detecta una caída en 2 a 4 s sin consultar de más
Máximo por nodo	30 peticiones	12 peticiones	Igual a los 12 procesos de PHP-FPM: el resto espera en la cola de HAProxy y no satura el nodo
Cabecera X-Servido-Por	Siempre	Solo en modo demostración	No mostrar nombres internos en el uso diario
Panel de estadísticas	admin / admin	Contraseña propia, sin versión, solo dentro del servidor	Desde el panel se puede apagar un nodo

5.3. Failover

En el Avance 3 apagamos la Arquitectura B con tráfico corriendo: HAProxy la marcó como DOWN en unos 2 segundos, todo el tráfico pasó al nodo A, no hubo ningún error en 39 peticiones y la sesión iniciada se mantuvo. Al volver a encenderla, B recibió tráfico otra vez a los 4 segundos. En el benchmark de la sección 6 los dos nodos atendieron más de 37.000 peticiones a través de HAProxy sin errores.

6. Pruebas de Rendimiento (Benchmarking)

6.1. Objetivo y método

Queríamos medir cuántas peticiones por segundo (RPS) atiende cada arquitectura y con qué latencia, y qué se gana al ponerlas juntas detrás de HAProxy. Para eso comparamos tres configuraciones: la **Arquitectura A** sola (pegándole directo a su nginx), la **Arquitectura B** sola y **A+B a través de HAProxy**.

Parámetro	Valor
Fecha	12/11/2026
Herramienta	ApacheBench (ab), en un contenedor dentro de la misma red de Docker que los nodos
Stack	El de producción (`docker-compose.prod.yml`): imágenes con OPcache, 12 procesos de PHP-FPM por nodo
Equipo	PC de desarrollo con Docker Desktop: 16 CPU lógicas y 8 GB de RAM para Docker. Los nodos, la base y el generador de carga comparten la misma máquina
Rutas	`/api/balanceo/ping`: liviana, sin consultas propias a la base. `/api/servicios`: el catálogo público, con consultas a la base
Carga	10 y 50 usuarios simultáneos; 3.000 peticiones por corrida en la ruta liviana y 1.500 en el catálogo
Repeticiones	3 corridas por combinación, después de 200 peticiones de calentamiento. Se informa la mediana
Total	36 corridas y 81.000 peticiones

6.2. Resultados

Ruta	Usuarios	Configuración	RPS	Latencia media	p95	p99	Errores
Ping	10	A	636	15,7 ms	21 ms	25 ms	0
Ping	10	B	612	16,3 ms	22 ms	25 ms	0
Ping	10	A+B (HAProxy)	606	16,5 ms	20 ms	22 ms	0
Ping	50	A	564	88,6 ms	105 ms	116 ms	0
Ping	50	B	517	96,7 ms	113 ms	121 ms	0
Ping	50	A+B (HAProxy)	567	88,2 ms	116 ms	132 ms	0
Catálogo	10	A	293	34,2 ms	42 ms	56 ms	0
Catálogo	10	B	287	34,8 ms	47 ms	53 ms	0
Catálogo	10	A+B (HAProxy)	293	34,1 ms	39 ms	44 ms	0
Catálogo	50	A	316	158,0 ms	180 ms	191 ms	0
Catálogo	50	B	291	172,0 ms	206 ms	235 ms	0
Catálogo	50	A+B (HAProxy)	353	141,7 ms	181 ms	205 ms	0

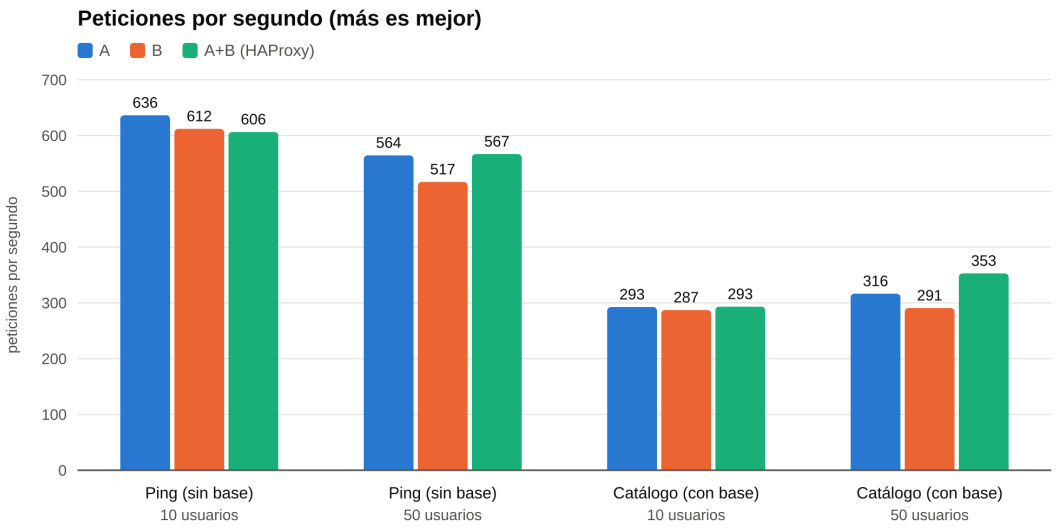


Figura 6. Peticiones por segundo de cada configuración (mediana de 3 corridas).

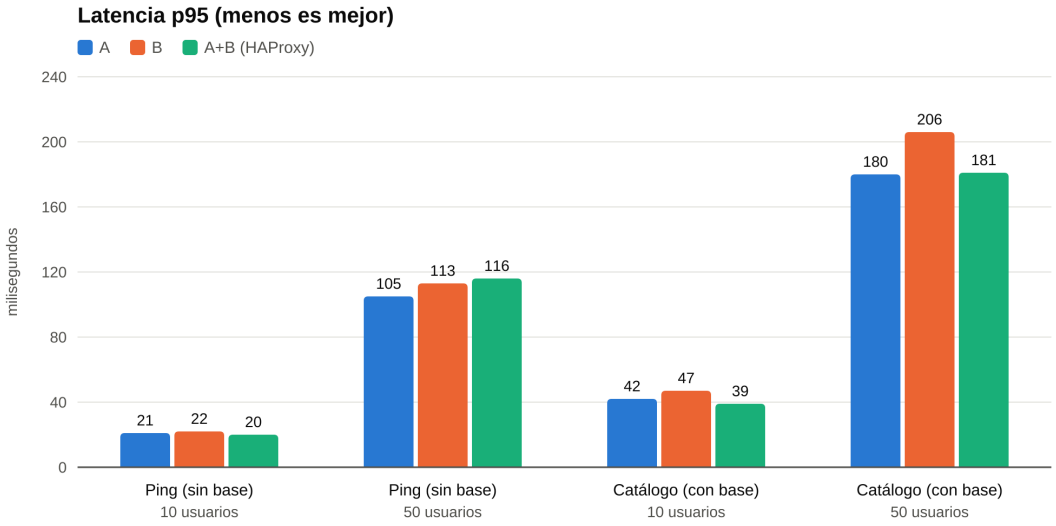


Figura 7. Latencia p95: el 95 % de las peticiones tardó eso o menos (mediana de 3 corridas).

6.3. Análisis

A y B rinden lo mismo. Las dos arquitecturas son la misma aplicación con los mismos recursos, y los resultados lo confirman: las diferencias entre A y B van de un 2 % a un 9 % y cambian de una corrida a otra. Que ninguna de las 81.000 peticiones fallara, incluso con 50 usuarios a la vez, muestra que las dos son estables.

Juntas rinden un poco más, pero no el doble. Con poca carga (10 usuarios) A+B atiende lo mismo que un nodo solo, porque a ningún nodo le falta capacidad. Con 50 usuarios en el catálogo, A+B atiende un 12 % más que A sola y baja la latencia media de 158 a 142 ms. En la ruta liviana no hay ganancia.

Dónde está el límite. Para entender por qué agregar un nodo casi no suma, medimos el uso de CPU de cada contenedor durante una carga de 50 usuarios sobre A+B:

Contenedor	Uso de CPU	Qué hace
db (PostgreSQL)	525 %	Base compartida por los dos nodos
app_b (PHP-FPM)	390 %	Nodo B
app (PHP-FPM)	383 %	Nodo A
haproxy	66 %	Balanceador
nginx y nginx_b	16 % cada uno	Servidores web

La base de datos es el contenedor que más CPU usa, aunque la ruta liviana no hace consultas propias. Al revisar las consultas activas encontramos el motivo: en cada petición Laravel verifica si el sitio está en modo mantenimiento, y lo configuramos para guardar ese dato en la caché de PostgreSQL (`APP_MAINTENANCE_DRIVER=cache`), para que los dos nodos lo compartan. Eso suma una consulta a la base por cada petición. Además, entre todos los contenedores se ocupan unas 14 de las 16 CPU de la máquina, así que agregar un nodo no puede sumar mucho: los dos nodos y la base compiten por el mismo procesador.

Conclusión. Con esta arquitectura, dos nodos dan tolerancia a fallos (si uno se cae, el otro sigue) y un poco más de capacidad con carga alta, pero el cuello de botella es la base de datos compartida. Para escalar de verdad, el siguiente paso sería mover la caché y el modo mantenimiento a Redis, que es más rápido para lecturas repetidas, y separar la base en otro servidor. Para el salón (tres personas atendiendo y sus clientas), los 290 a 350 pedidos por segundo que atiende el catálogo están muy por encima de lo que necesita.

6.4. Repetición en el VPS (trabajo futuro)

El sistema ya está publicado en el VPS de Contabo (4 vCPU y 8 GB de RAM), pero el benchmark no se repitió ahí antes de la entrega: correr pruebas de carga sobre el servidor de producción, que ya usa el salón, podía afectar a la dueña y a sus clientas. Queda como trabajo futuro, con este comando desde el mismo servidor:

```
docker run --rm --network menina_prod_web httpd:2.4-alpine \
  ab -n 1500 -c 50 http://haproxy/api/servicios
```

7. Dashboard HAProxy Stats con Tráfico Real

El panel de estadísticas de HAProxy escucha en el puerto 8404, ruta `/stats`, con usuario y contraseña. En producción solo se abre dentro del servidor: se entra con un túnel SSH (`ssh -L 8404:127.0.0.1:8404 deploy@IP_DEL_VPS`) y se navega a `http://localhost:8404/stats`.

Esta captura se tomó al terminar el benchmark de la sección 6 y las mediciones de CPU, con todo el tráfico que pasó por HAProxy en esas pruebas:

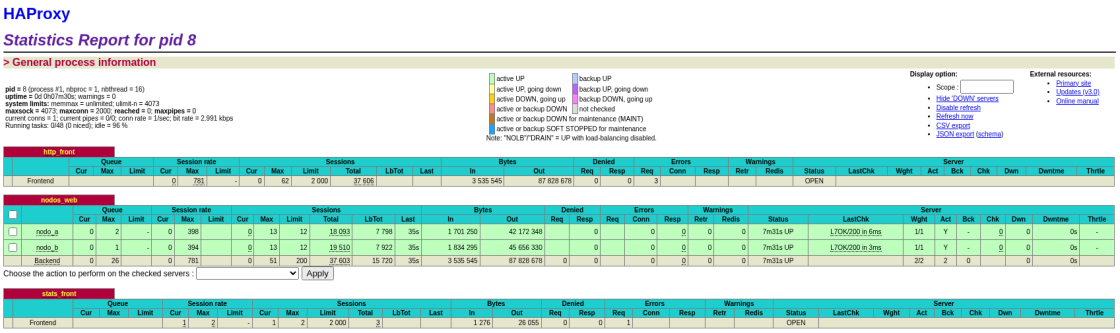


Figura 8. Panel HAProxy Stats (puerto 8404) después del benchmark, con los dos nodos en UP.

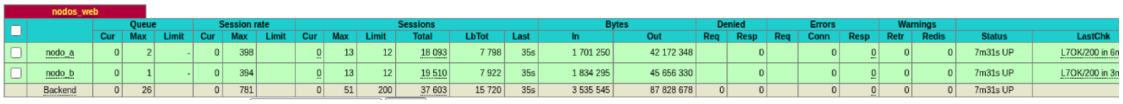


Figura 9. Detalle del backend nodos_web en el mismo panel: sesiones atendidas por cada nodo, cola y errores.

Dato del panel	nodo_a	nodo_b	Total del backend
Sesiones atendidas	18.093	19.510	37.603
Máximo de sesiones al mismo tiempo	13	13	51
Máximo en cola de HAProxy	2	1	26
Errores de conexión y de respuesta	0	0	0
Estado y último chequeo	UP · L7OK/200 en 6 ms	UP · L7OK/200 en 3 ms	2 de 2 nodos activos

Se ve que leastconn repartió la carga casi en partes iguales (48 % y 52 %) y que, cuando llegaron más de 12 peticiones juntas a un nodo, las que sobraban esperaron en la cola de HAProxy (hasta 26) en lugar de saturar a PHP-FPM. El pico fue de 781 sesiones nuevas por segundo en la entrada.

Para la exposición también está la página `/balanceo` (solo con el modo demostración activo), que manda peticiones reales a través de HAProxy y muestra en vivo qué nodo atendió cada una:

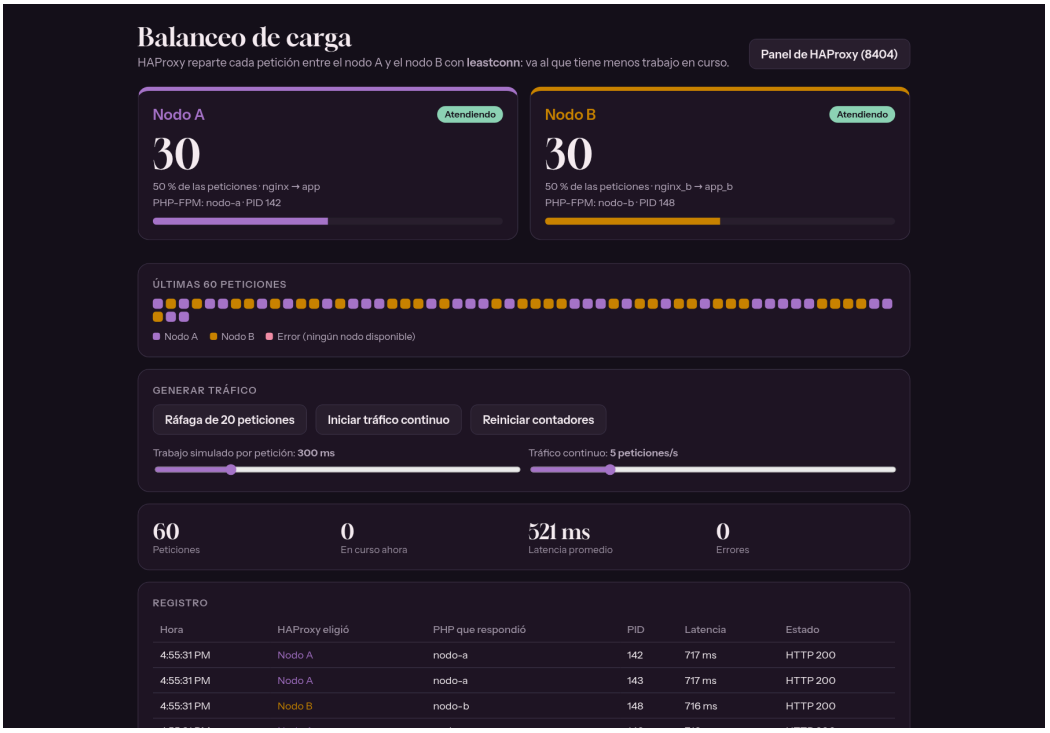


Figura 10. Página /balanceo después de tres ráfagas de 20 peticiones: 30 las atendió A y 30 B.

8. Despliegue en Producción

El sistema está publicado en un **Contabo Cloud VPS 4** (4 vCPU, 8 GB de RAM, Ubuntu 24.04). Todo corre con Docker Compose en un solo servidor y se instala desde el repositorio de GitHub; en el servidor solo se crea a mano el archivo con las contraseñas.

8.1. Arquitectura de producción

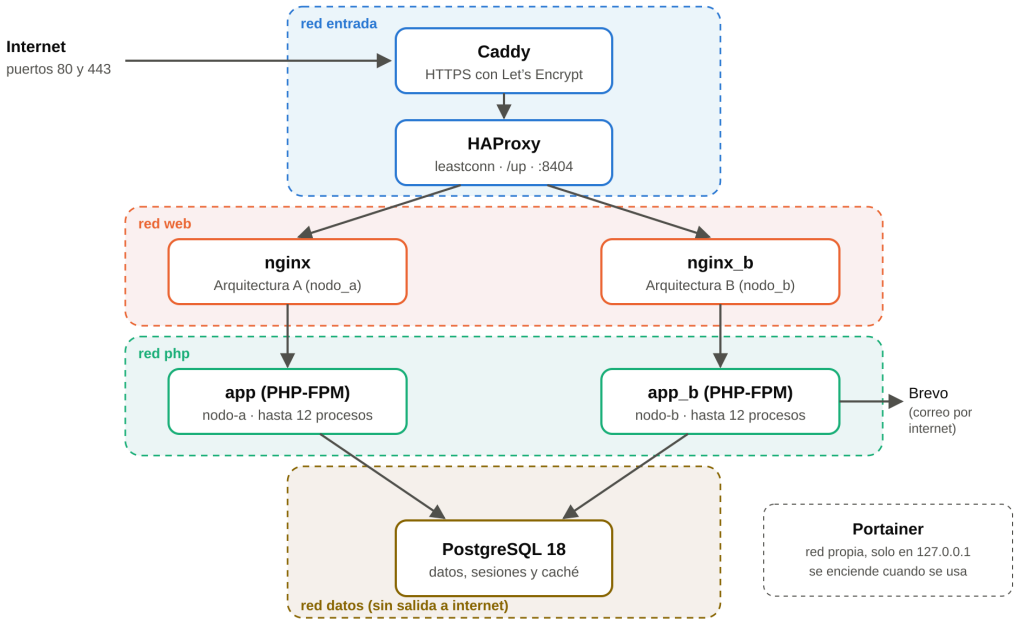


Figura 11. Arquitectura de producción: cada tramo va en su propia red de Docker.

Solo Caddy recibe tráfico de internet (puertos 80 y 443). Caddy pide y renueva solo el certificado de Let's Encrypt, redirige http a https y le pasa todo a HAProxy. Cada tramo va en una red de Docker distinta: Caddy y HAProxy no llegan a la base, y la base no tiene salida a internet. Portainer, que controla Docker, va en su propia red y se enciende solo cuando se usa.

8.2. Componentes

Archivo	Para qué sirve
`Dockerfile.prod`	Imágenes multi-etapa: dependencias de Composer sin las de desarrollo, build del frontend con Node y una imagen final de PHP-FPM sin Node, Git ni Composer. nginx solo recibe la carpeta `public`
`docker-compose.prod.yml`	Los servicios de producción, con redes separadas y `no-new-privileges` en todos los contenedores
`deploy/caddy/Caddyfile`	HTTPS, HSTS y redirección de `www`
`deploy/haproxy/haproxy.cfg`	Balanceo, health checks y panel de estadísticas
`deploy/nginx/`	nginx de cada nodo con cabeceras de seguridad
`deploy/postgres/`	Crea el usuario de la aplicación sin permisos de superusuario

Archivo	Para qué sirve
`deploy/deploy.sh`	Despliegue y actualización: valida la configuración, baja imágenes nuevas, construye, migra una sola vez y comprueba que el sitio responda
`deploy/primer-arranque.sh`	Primera vez: despliegue, catálogo real del salón y cuenta de la administración
`deploy/backup.sh` y `restaurar.sh`	Copias de la base y restauración
`deploy/README.md`	Guía paso a paso: servidor, firewall, DNS o sslip.io, correo, cuentas del equipo, actualizaciones y problemas comunes

8.3. Procedimiento

1. Preparar el servidor: acceso solo con llave SSH, sin login de root, firewall con los puertos 22, 80 y 443, fail2ban, actualizaciones automáticas y Docker.
2. Apuntar el dominio a la IP del VPS. Sin dominio propio se usa `IP-DEL-VPS.sslip.io`, que también tiene HTTPS.
3. Clonar el repositorio en `/opt/menina` y crear `deploy/.env.production` con las contraseñas, el dominio y los datos de Brevo.
4. Generar la clave de la aplicación y correr `bash deploy/primer-arranque.sh`.
5. Cargar el personal desde el panel y crear las cuentas del equipo desde la terminal con `php artisan salon:crear-cuenta`.
6. Para actualizar más adelante: unir los cambios a `main` y correr `bash deploy/deploy.sh` en el servidor.

8.4. Estado del despliegue

Dato	Valor
URL pública	https://176.57.189.106.sslip.io
Fecha del primer despliegue	12/11/2026
Prueba de correo (`correo:probar`)	Correo recibido correctamente



Figura 12. Correo de prueba enviado desde el servidor de producción con Brevo (`correo:probar`).

9. Seguridad

Antes de pasar a producción hicimos una auditoría de seguridad de todo el sistema, asumiendo que cualquiera puede leer el código (el repositorio es público). La revisión se dividió en tres frentes: autenticación y permisos, datos de entrada y salida, e infraestructura y despliegue. Cada hallazgo se verificó en el código antes de corregirlo. Las correcciones de la aplicación tienen su prueba automatizada, y las de infraestructura las comprobamos levantando el stack de producción (sección 9.2).

9.1. Hallazgos principales y correcciones

Hallazgo	Gravedad	Corrección
Con la cabecera `X-Forwarded-Prefix`, el enlace de "Olvidé mi contraseña" podía apuntar a otro dominio y el token terminaba en manos de un atacante	Alta	El enlace se arma con `APP_URL`; Laravel solo confía en las cabeceras que reescribe Caddy, y Caddy borra las demás
Pedidos de cita sin límite: con un script se llenaba la bandeja y se agotaba la cuota de correos	Alta	Hasta 5 pedidos esperando respuesta por clienta y 10 cada 10 minutos
Una profesional sin ficha veía las reservas de todo el salón	Media	Responde 403
El listado de clientas mostraba correo, dirección y ubicación a profesionales sin ese permiso	Media	Sin el permiso solo llegan nombre y teléfono
Una profesional dada de baja seguía entrando	Media	Pierde la sesión en la siguiente petición y el login la rechaza
Restablecer la contraseña no cerraba las otras sesiones	Media	Se cierran todas
Una reserva podía cargarse con cantidad 0 o negativa	Media	Solo se aceptan servicios por id, sin repetir
El texto de la clienta podía simular un enlace del salón dentro del correo a la dueña	Media	Va en una sola línea, al final y entre comillas
La aplicación entraba a PostgreSQL como superusuario	Media	Usuario propio, dueño solo de su base; el superusuario queda para los backups
Todos los contenedores en una misma red; Portainer siempre encendido	Media	Redes separadas, `no-new-privileges` y Portainer bajo demanda
La API no tenía protección CSRF	Baja	Protección activada; el frontend manda el token
Dependencias con avisos de seguridad (Symfony, Vite)	Media	Actualizadas: `composer audit` y `npm audit` sin vulnerabilidades

9.2. Verificación

Además de las pruebas automatizadas, levantamos el stack de producción completo y comprobamos: que Caddy y nginx no llegan a la base y la base no sale a internet; que el usuario de la aplicación no puede ejecutar comandos del sistema (`COPY ... TO PROGRAM`), leer archivos del servidor, crear bases ni crear usuarios; que Laravel ve la IP real de cada persona y no la de Caddy; que las cabeceras de seguridad llegan al navegador; y que el backup y la restauración funcionan con el usuario nuevo. Para las

correcciones más graves también verificamos que las pruebas fallan con el código anterior y pasan con el nuevo.

10. Evidencia de Pruebas

10.1. Pruebas automatizadas

La suite de PHPUnit pasó de 19 pruebas en el Avance 2 y 191 en el Avance 3 a **232 pruebas en 15 archivos**:

Archivo	Pruebas	Qué cubre
ReservaTest	52	Reglas de negocio, estados, modificación, cancelación y validación de servicios
SolicitudCitaTest	36	Pedidos de cita, bandeja de la dueña, topes, avisos por correo y condiciones de carrera
AutenticacionTest	32	Login, registro, recuperación de contraseña, CSRF y enlaces seguros
ProduccionTest	18	Configuración de producción, seeders y comandos de cuentas del equipo
PermisoTest	14	Permisos por profesional y bajas del personal
AgendaAjustesTest	14	Horarios ofrecidos, fuera de horario y cierres por rango
SprintAvanceTest	13	Criterios de aceptación de los avances anteriores
EspecialidadHorarioTest	12	Especialidades y horario del salón
ExpedienteTest	10	Ficha, preferencias e historial de clientas
PagoTest	9	Seña, pago final y saldo
SalonPublicoTest	8	Página pública y catálogo
ReporteTest	7	Reportes de reservas, servicios e ingresos
BalanceoTest	5	Página /balanceo y ruta de ping
ExampleTest (Feature y Unit)	2	Pruebas base de Laravel

Con `docker compose exec app php artisan test` pasan las 232 pruebas (1.655 aserciones) en unos 30 segundos. El frontend se compila con `npm run build` sin errores.

10.2. Pruebas sobre la infraestructura

- **Failover** (Avance 3): un nodo apagado con tráfico corriendo, 0 errores y la sesión intacta.
- **Benchmark** (sección 6): 81.000 peticiones contra A, B y A+B, 0 errores.
- **Stack de producción** (sección 9.2): HTTPS, aislamiento de redes, permisos del usuario de la base, backup y restauración.
- **Correo:** envío real por Brevo con `php artisan correo:probar`.

11. Repositorio Git y Documentación

11.1. Repositorio

El código está en GitHub: **https://github.com/Wycie-R/meninaCoiffure-back**. Al 13/11 la rama `main` tiene 90 commits y se unieron 17 Pull Requests.

- **Ramas:** `main` es siempre la versión estable, la que se despliega. Cada cambio se hace en una rama corta y entra por Pull Request.
- **Commits:** en español y en primera persona, con un título corto que dice qué cambia y un cuerpo que explica por qué.
- **Secretos:** ningún `.env` real está en Git; solo se versionan las plantillas (`src/.env.example` y `deploy/.env.production.example`). Revisamos todo el historial y no hay contraseñas reales.

11.2. Estructura

Carpeta o archivo	Contenido
`src/`	La aplicación Laravel: `app/` (controladores, modelos, servicios), `database/` (migraciones y seeders), `resources/js/` (frontend Vue), `routes/` y `tests/`
`deploy/`	Todo lo de producción: configuraciones, scripts y la guía `deploy/README.md`
`docker-compose.yml`, `Dockerfile`, `haproxy.cfg`, `nginx.conf`	Entorno de desarrollo
`docker-compose.prod.yml`, `Dockerfile.prod`	Entorno de producción
`documentos/`	Entregas de los avances, diccionario de datos y diagrama entidad-relación
`portainer/`	Guía de Portainer

11.3. Documentación disponible

Documento	Dónde
Instalación del entorno de desarrollo	`README.md` (secciones Instalación y Actualización)
Despliegue y operación en producción	`deploy/README.md`
API REST	Swagger UI en `/docs` (solo para la administración); especificación en `src/resources/openapi/openapi.json`
Modelo de datos	`documentos/der_diccionario_datos_menina_coiffure.pdf` y `documentos/der.html`
Metodología	Reporte de los Sprints 1 y 2 y los documentos de los Avances 2, 3 y 4
Alcance y caso de negocio	Documento del Avance 1 (`documentos/docAvance1.docx`)

12. Ethical Hacking | Test de Intrusión Externo | Web

12.1. Objetivo

El servicio de consultoría de «Ethical Hacking | Test de Intrusión Externo | Web» de Ciberseguridad tiene por objetivo principal determinar las debilidades existentes y recomendar las mejores soluciones posibles para eliminarlas o minimizarlas, y de esta forma incrementar el grado de seguridad de la infraestructura externa expuesta a Internet y de las aplicaciones web de Menina Coiffure.

La modalidad utilizada para el testeo de la infraestructura externa fue **BlackBox** (sin credenciales de acceso).

12.2. Alcance

El alcance definido por Menina Coiffure fue el siguiente:

Ítem	Detalle
Dirección IP pública	176.57.189.106
Dominio	176.57.189.106.sslip.io

12.3. Personal comprometido

Integrante	Rol
Giovanni Rojas	Líder · Backend & Software Architect Lead · DevOps & Cloud Engineer
Noelia Servián	Backend & Software Architect Lead· DevOps & Cloud Engineer
Lujan Ramírez	Scrum Master / Project Manager · Frontend & QA Engineer

12.4. Duración

El test se realizó entre los días 11 y 12 de noviembre de 2026.

12.5. Herramientas utilizadas

Las herramientas utilizadas para la ejecución del test fueron las siguientes:

Herramienta	Uso en el test
Nmap	Descubrimiento de puertos abiertos y servicios expuestos
Burp Suite	Análisis de las peticiones y respuestas de las aplicaciones web
DIRB	Búsqueda de rutas y directorios no publicados
Nikto	Detección de configuraciones inseguras en el servidor web
Nessus Vulnerability Scanner	Escaneo de vulnerabilidades conocidas

12.6. Contexto del test de intrusión

El objetivo de este trabajo fue analizar las funcionalidades que se encuentran publicadas en la infraestructura externa y web expuesta a Internet de Menina Coiffure. El análisis tenía la finalidad de descubrir y explotar vulnerabilidades o errores dentro de las siguientes áreas:

- Control de acceso
- Autenticación
- Configuración
- Cifrado
- Protección de datos
- Manejo de errores

Teniendo en cuenta dichas áreas, las tareas de búsqueda y comprobación de errores y/o vulnerabilidades que fueron parte del testeo incluyen (pero no se limitaron a) pruebas de:

- Divulgación de información
- Elevación de privilegios
- Protección de datos
- Actualización de componentes de software (patching)
- Análisis criptográfico
- SQL Injection
- Cross Site Scripting

El siguiente cuadro resume los puertos expuestos a Internet del alcance definido (host 176.57.189.106.sslip.io, IP 176.57.189.106):

Puerto	**Protocolo**	**Estado**	**Servicio**
22	TCP	Abierto	SSH (administración del servidor)
80	TCP	Abierto	HTTP: solo redirige a HTTPS
443	TCP	Abierto	HTTPS (Caddy)

El puerto 22 es el acceso de administración: solo acepta llaves SSH, sin contraseña ni login de root, y está protegido con fail2ban (sección 8.3). Los puertos 80 y 443 los atiende Caddy, y el 80 solo redirige a HTTPS.

12.7. App Web Institucional

Página pública del salón (`/salon`): catálogo, promociones, horario y acceso a las cuentas.

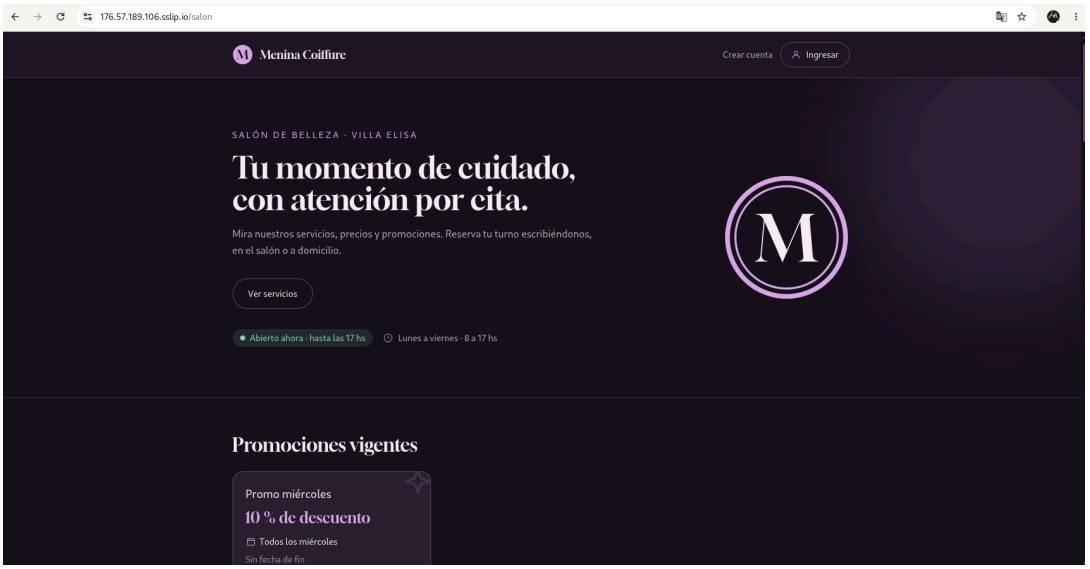


Figura 13. App Web Institucional: página pública del salón en producción.

12.8. App Web de Clientas

Registro y cuenta de las clientas (`/registro` y `/mi-cuenta`), donde cada clienta crea su usuario y pide sus citas.

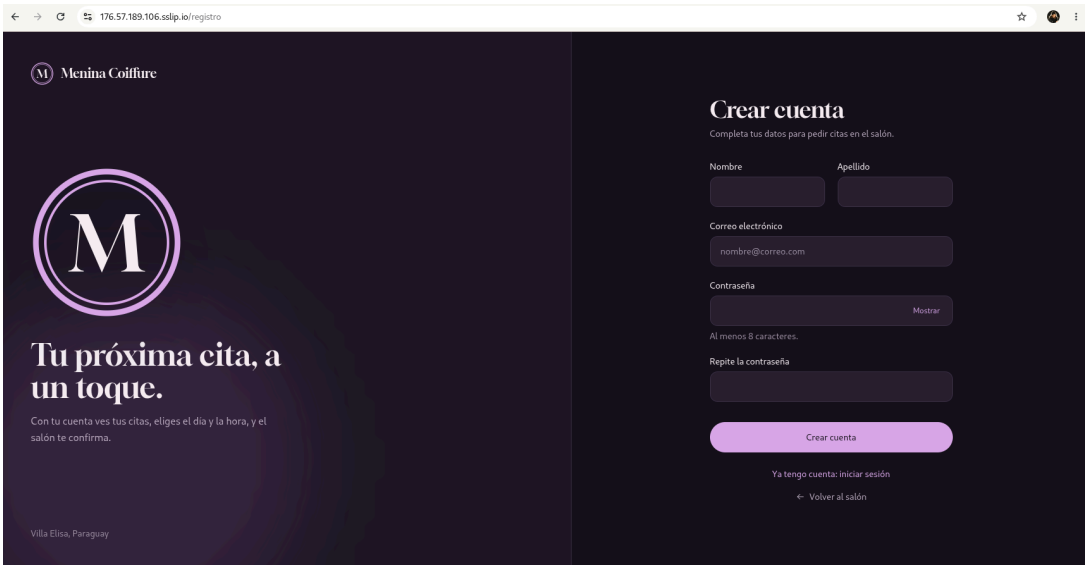


Figura 14. App Web de Clientas: registro de una cuenta nueva en producción.

12.9. Resultados

No se identificaron vulnerabilidades durante el análisis realizado. El escaneo de Nessus con la plantilla «Pentest Externo v2» terminó sin vulnerabilidades críticas, altas, medias ni bajas:

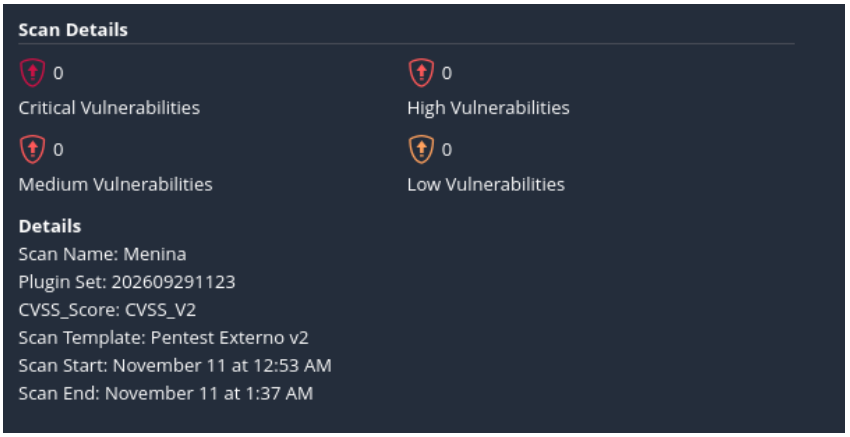


Figura 15. Resumen del escaneo de Nessus del 11/11/2026: 0 vulnerabilidades en todas las categorías.

13. Conclusiones

Producto. Empezamos con un CRUD de reservas y terminamos con un sistema completo para el salón y sus clientas: agenda con reglas de negocio, pagos, reportes, página pública y cuentas para pedir citas, con 232 pruebas automatizadas que lo respaldan. Reemplazar KeyCloak por la autenticación de Laravel simplificó la infraestructura sin perder funcionalidad.

Infraestructura. HAProxy reparte el tráfico entre las arquitecturas A y B con leastconn, revisa los nodos con health checks y da tolerancia a fallos: si un nodo se cae, el otro sigue sin que el usuario lo note. El benchmark mostró que las dos arquitecturas rinden lo mismo y que juntas ganan algo de capacidad con carga alta, pero que el límite real es la base de datos compartida; el siguiente paso para escalar sería Redis.

Seguridad y producción. El sistema está publicado en un servidor real con HTTPS automático, redes separadas, un usuario de base sin privilegios de más, contraseñas que no pasan por Git y una guía para desplegarlo y mantenerlo. El test de intrusión externo no encontró vulnerabilidades (sección 12).

Proceso. El cambio que más mejoró el flujo fue integrar cada tarea con su propio Pull Request: el trabajo dejó de acumularse en revisión y el Cycle Time bajó de 8 días a menos de 3. Lo que más nos costó fue repartir el trabajo a lo largo del sprint en lugar de concentrarlo al final; para un próximo proyecto acordamos respetar el WIP Limit desde el primer sprint y dejar un acta breve de cada ceremonia.

14. Referencias

Anderson, D. J. (2010). *Kanban: Successful evolutionary change for your technology business*. Blue Hole Press.

Apache Software Foundation. (s.f.). *ab: Apache HTTP server benchmarking tool*. https://httpd.apache.org/docs/2.4/programs/ab.html

Brevo. (s.f.). *Documentación de SMTP*. https://help.brevo.com

Caddy. (s.f.). *Caddy 2 documentation: Automatic HTTPS y reverse_proxy*. https://caddyserver.com/docs

Docker Inc. (s.f.). *Docker Compose y networking*. https://docs.docker.com

HAProxy Technologies. (s.f.). *HAProxy configuration manual 3.0*. https://docs.haproxy.org

Inertia.js. (s.f.). *Documentación*. https://inertiajs.com

Laravel. (s.f.). *Documentación oficial: Authentication, Resetting Passwords y Deployment*. https://laravel.com/docs

Lyon, G. (s.f.). *Nmap reference guide*. https://nmap.org/book/man.html

OWASP Foundation. (2021). *OWASP Top 10*. https://owasp.org/Top10

Portainer. (s.f.). *Portainer documentation*. https://docs.portainer.io

PortSwigger. (s.f.). *Burp Suite documentation*. https://portswigger.net/burp/documentation

PostgreSQL Global Development Group. (s.f.). *PostgreSQL 18 documentation*. https://www.postgresql.org/docs

Schwaber, K. y Sutherland, J. (2020). *The Scrum guide*. https://scrumguides.org

Sullo, C. y Lodge, D. (s.f.). *Nikto web server scanner*. https://github.com/sullo/nikto

Tenable. (s.f.). *Nessus documentation*. https://docs.tenable.com/nessus

The Dark Raver. (s.f.). *DIRB: Web content scanner*. https://dirb.sourceforge.net

Vue.js. (s.f.). *Guía oficial*. https://vuejs.org/guide